```
// ===== FILE BEGIN: screen-catch\api.js =====
import path from "node:path";
import fs from "node:fs";
import { spawn } from "node:child_process";
import { fileURLToPath } from "node:url";
import { setupAsrSocket } from "./asr.js";
export function setupScreenCatchRoutes(app, server) {
  const dataRootDir = path.resolve(process.cwd(), "..", "screen-catch", "data");
  const __filename = fileURLToPath(import.meta.url);
  const __dirname = path.dirname(__filename);
  const finalizeScript = path.join(__dirname, "asr_finalize.py");
  const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
  const ensureWithinDataRoot = (targetPath) => {
    const resolved = path.resolve(dataRootDir, targetPath);
    return resolved.startsWith(dataRootDir) ? resolved : null;
  };
  const formatSpeakerLabel = (rawSpeakerId, speakerMap) => {
    if (rawSpeakerId === undefined || rawSpeakerId === null || rawSpeakerId === "") {
      return "未知发言人";
    }
    const raw = String(rawSpeakerId);
    if (!speakerMap.has(raw)) {
      speakerMap.set(raw, speakerMap.size + 1);
    }
    return `发言人${speakerMap.get(raw)}`;
  };
  const formatTime = (ms = 0) => {
    const totalSeconds = Math.max(0, Math.floor(Number(ms || 0) / 1000));
    const hh = String(Math.floor(totalSeconds / 3600)).padStart(2, "0");
    const mm = String(Math.floor((totalSeconds % 3600) / 60)).padStart(2, "0");
    const ss = String(totalSeconds % 60).padStart(2, "0");
    return `${hh}:${mm}:${ss}`;
  };
  app.post("/api/screenshot", (req, res) => {
    try {
      const { image, sessionId } = req.body;
      if (!image) return res.status(400).json({ ok: false, error: "missing_image" });
      const sessionDirName = sessionId || "default-session";
      const baseDir = path.resolve(process.cwd(), "..", "screen-catch", "data", sessionDirName);
      const picsDir = path.join(baseDir, "pics");
      if (!fs.existsSync(picsDir)) {
        fs.mkdirSync(picsDir, { recursive: true });
      }
      const base64Data = image.replace(/^data:image\/\w+;base64,/, "");
      const buffer = Buffer.from(base64Data, "base64");
      const timestamp = new Date().toISOString().replace(/[:.]/g, "-");
      const filename = `screenshot-${timestamp}.png`;
      const filepath = path.join(picsDir, filename);
      fs.writeFileSync(filepath, buffer);
      res.json({ ok: true, filename });
```

```
  } catch (e) {
    console.error("Screenshot save error:", e);
    res.status(500).json({ ok: false, error: "server_error" });
  }
});
app.get("/api/asr/files/*", (req, res) => {
  try {
    const relativePath = req.params[0];
    const filePath = ensureWithinDataRoot(relativePath || "");
    if (!filePath || !fs.existsSync(filePath)) {
      return res.status(404).json({ ok: false, error: "file_not_found" });
    }
    res.sendFile(filePath);
  } catch (e) {
    console.error("ASR file serve error:", e);
    res.status(500).json({ ok: false, error: "server_error" });
  }
});
app.post("/api/asr/finalize", async (req, res) => {
  try {
    const { sessionId, audioUrl } = req.body || {};
    if (!sessionId && !audioUrl) {
      return res.status(400).json({ ok: false, error: "missing_sessionId_or_audioUrl" });
    }
    let audioSource = audioUrl || "";
    let transcriptFile = "";
    if (sessionId) {
      const sessionDir = ensureWithinDataRoot(sessionId);
      if (!sessionDir) {
        return res.status(400).json({ ok: false, error: "invalid_sessionId" });
      }
      const wavAudioFile = path.join(sessionDir, "full-audio.wav");
      transcriptFile = path.join(sessionDir, "transcript.txt");
      for (let i = 0; i < 20 && !fs.existsSync(wavAudioFile); i += 1) {
        await wait(500);
      }
      if (!fs.existsSync(wavAudioFile)) {
        return res.status(404).json({ ok: false, error: "full_audio_not_found" });
      }
      if (!audioSource) {
        const publicBase = (process.env.ASR_AUDIO_PUBLIC_BASE_URL || "").trim().replace(/\/$/, "");
        if (!publicBase) {
          return res.status(400).json({
            ok: false,
            error: "missing_ASR_AUDIO_PUBLIC_BASE_URL",
            message: "请配置可公网访问当前服务的 ASR_AUDIO_PUBLIC_BASE_URL，供百炼拉取 full-audio.wav"
          });
        }
        audioSource = `${publicBase}/api/asr/files/${encodeURIComponent(sessionId)}/full-audio.wav`;
      }
```

```
}
const pythonArgs = [finalizeScript, audioSource];
const finalizeResult = await new Promise((resolve, reject) => {
  const child = spawn("python", pythonArgs, {
    stdio: ["ignore", "pipe", "pipe"],
    env: {
      ...process.env,
      PYTHONUTF8: "1",
      PYTHONIOENCODING: "utf-8"
    }
  });
  let stdout = "";
  let stderr = "";
  child.stdout.on("data", (chunk) => {
    stdout += chunk.toString();
  });
  child.stderr.on("data", (chunk) => {
    stderr += chunk.toString();
  });
  child.on("error", reject);
  child.on("close", (code) => {
    try {
      const parsed = stdout.trim() ? JSON.parse(stdout.trim()) : null;
      resolve({ code, parsed, stderr: stderr.trim(), stdout: stdout.trim() });
    } catch (err) {
      reject(new Error(`finalize_json_parse_failed: ${stdout || stderr}`));
    }
  });
});
const { code, parsed, stderr } = finalizeResult;
if (code !== 0 || !parsed?.ok) {
  return res.status(500).json({
    ok: false,
    error: parsed?.error || "finalize_failed",
    message: parsed?.message || stderr || "离线说话人分离失败",
    model: parsed?.model,
    audioUrl: parsed?.audioUrl || audioSource
  });
}
const speakerMap = new Map();
const transcriptEntries = (parsed.sentences || []).map((sentence) => {
  const speaker = formatSpeakerLabel(sentence.speakerId, speakerMap);
  return {
    speaker,
    text: sentence.text || "",
    beginTime: Number(sentence.beginTime || 0),
    endTime: Number(sentence.endTime || 0)
  };
}).filter((entry) => entry.text);
if (transcriptFile) {
```

```
      const transcriptText = transcriptEntries.map((entry) => `[${entry.speaker}] ${entry.text}`).join("\n");
      fs.writeFileSync(transcriptFile, transcriptText ? `${transcriptText}\n` : "", "utf8");
    }
    res.json({
      ok: true,
      data: {
        model: parsed.model,
        audioUrl: parsed.audioUrl || audioSource,
        transcriptionUrl: parsed.transcriptionUrl || "",
        lines: transcriptEntries.map((entry) => `[${formatTime(entry.beginTime)}] [${entry.speaker}] ${entry.text}`),
        sentences: transcriptEntries
      }
    });
  } catch (e) {
    console.error("ASR finalize error:", e);
    res.status(500).json({ ok: false, error: "server_error", message: e.message || "unknown_error" });
  }
 });
 setupAsrSocket(server);
}
// ===== FILE END =====
// ===== FILE BEGIN: screen-catch\asr.js =====
import { WebSocketServer } from "ws";
import fs from "node:fs";
import path from "node:path";
import { spawn } from "node:child_process";
import readline from "node:readline";
import { fileURLToPath } from "node:url";
export function setupAsrSocket(server) {
 const wss = new WebSocketServer({ server, path: "/ws/asr" });
 const __filename = fileURLToPath(import.meta.url);
 const __dirname = path.dirname(__filename);
 const bridgeScript = path.join(__dirname, "asr_bridge.py");
 const createWavHeader = (dataSize, sampleRate = 16000, channels = 1, bitsPerSample = 16) => {
  const header = Buffer.alloc(44);
  const byteRate = sampleRate * channels * bitsPerSample / 8;
  const blockAlign = channels * bitsPerSample / 8;
  header.write("RIFF", 0);
  header.writeUInt32LE(36 + dataSize, 4);
  header.write("WAVE", 8);
  header.write("fmt ", 12);
  header.writeUInt32LE(16, 16);
  header.writeUInt16LE(1, 20);
  header.writeUInt16LE(channels, 22);
  header.writeUInt32LE(sampleRate, 24);
  header.writeUInt32LE(byteRate, 28);
  header.writeUInt16LE(blockAlign, 32);
  header.writeUInt16LE(bitsPerSample, 34);
  header.write("data", 36);
  header.writeUInt32LE(dataSize, 40);
```

```
  return header;
};
wss.on("connection", (ws, req) => {
  console.log("Client connected to ASR WebSocket");
  let sessionId = "default-session";
  if (req && req.url) {
    try {
      const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
      sessionId = url.searchParams.get("sessionId") || "default-session";
    } catch(e) {}
  }
  const apiKey = process.env.QWEN_API_KEY;
  const safeWsSend = (obj) => {
    if (ws.readyState === 1) { // WebSocket.OPEN
      try {
        ws.send(JSON.stringify(obj));
      } catch (e) {
        console.error("ws.send error:", e);
      }
    }
  };
  if (!apiKey) {
    safeWsSend({ type: "error", message: "Missing QWEN_API_KEY in .env" });
    ws.close();
    return;
  }
  const baseDir = path.resolve(process.cwd(), "..", "screen-catch", "data", sessionId);
  if (!fs.existsSync(baseDir)) {
    fs.mkdirSync(baseDir, { recursive: true });
  }
  const transcriptFile = path.join(baseDir, "transcript.txt");
  const rawAudioFile = path.join(baseDir, "full-audio.pcm");
  const wavAudioFile = path.join(baseDir, "full-audio.wav");
  fs.writeFileSync(transcriptFile, "", "utf8");
  const audioWriteStream = fs.createWriteStream(rawAudioFile, { flags: "w" });
  console.log("ASR transcript target:", transcriptFile);
  let asrProcess = null;
  let sentenceWritten = false;
  let latestPartialText = "";
  let fallbackFlushed = false;
  let audioFinalized = false;
  const speakerMap = new Map();
  let nextSpeakerNumber = 1;
  const mapSpeakerLabel = (rawSpeakerId) => {
    if (rawSpeakerId === undefined || rawSpeakerId === null || rawSpeakerId === "") {
      return "未知发言人";
    }
    const raw = String(rawSpeakerId);
    if (!speakerMap.has(raw)) {
      speakerMap.set(raw, nextSpeakerNumber);
```

```
      nextSpeakerNumber += 1;
    }
    return `发言人${speakerMap.get(raw)}`;
  };
  const sanitizeText = (text) => {
    if (typeof text !== "string") return "";
    const cleaned = text
      .replace(/[\u0000-\u0008\u000B\u000C\u000E-\u001F\u007F]/g, "")
      .replace(/[^A-Za-z0-9\u4e00-\u9fff\s.,!?;:'"()\-，。！？；：、（）]/g, "")
      .replace(/\s+/g, " ")
      .trim();
    if (!/[A-Za-z0-9\u4e00-\u9fff]/.test(cleaned)) return "";
    return cleaned.length > 2000 ? cleaned.slice(0, 2000) : cleaned;
  };
  const finalizeAudioFile = () => {
    if (audioFinalized) return;
    audioFinalized = true;
    if (!fs.existsSync(rawAudioFile)) return;
    let dataSize = 0;
    try {
      dataSize = fs.statSync(rawAudioFile).size;
    } catch (err) {
      console.error("Error stating raw audio file:", err);
      return;
    }
    const wavStream = fs.createWriteStream(wavAudioFile, { flags: "w" });
    wavStream.on("error", (err) => {
      console.error("Error writing wav audio file:", err);
    });
    wavStream.write(createWavHeader(dataSize));
    const rawReadStream = fs.createReadStream(rawAudioFile);
    rawReadStream.on("error", (err) => {
      console.error("Error reading raw audio file:", err);
      wavStream.destroy(err);
    });
    wavStream.on("finish", () => {
      console.log("ASR wav finalized:", wavAudioFile, "pcmBytes=", dataSize);
    });
    rawReadStream.pipe(wavStream);
  };
  const flushFallbackText = () => {
    if (fallbackFlushed) return;
    if (sentenceWritten) return;
    const text = sanitizeText(latestPartialText || "");
    if (!text) return;
    fallbackFlushed = true;
    const speakerLabel = "未知发言人";
    const lineText = `[${speakerLabel}] ${text}\n`;
    try {
      fs.appendFileSync(transcriptFile, lineText, "utf8");
```

```
      console.log("ASR fallback transcript bytes:", Buffer.byteLength(lineText, "utf8"));
      safeWsSend({ type: "sentence", text, speakerId: speakerLabel });
    } catch (err) {
      console.error("Error writing fallback transcript file:", err);
    }
  };
  try {
    asrProcess = spawn("python", ["-u", bridgeScript], {
      stdio: ["pipe", "pipe", "pipe"],
      env: {
        ...process.env,
        QWEN_API_KEY: apiKey,
        ASR_MODEL: "paraformer-realtime-v2",
        ASR_FORMAT: "pcm",
        ASR_SAMPLE_RATE: "16000",
        PYTHONUTF8: "1",
        PYTHONIOENCODING: "utf-8"
      }
    });
    asrProcess.stdin.on("error", (err) => {
      if (err.code === "EPIPE" || err.code === "EOF") {
        // Ignore EPIPE/EOF errors when python process closes its stdin
      } else {
        console.error("ASR process stdin error:", err);
      }
    });
    audioWriteStream.on("error", (err) => {
      console.error("ASR raw audio stream error:", err);
    });
    const outputReader = readline.createInterface({ input: asrProcess.stdout });
    outputReader.on("line", (line) => {
      try {
        if (!line) return;
        let msg;
        try {
          msg = JSON.parse(line);
        } catch {
          return;
        }
        const type = msg?.type;
        if (type === "ready") {
          safeWsSend({ type: "ready" });
          console.log("Python ASR ready");
          return;
        }
        if (type === "partial") {
          const text = sanitizeText(msg.text || "");
          if (text) {
            latestPartialText = text;
          }
```

```
        safeWsSend({
          type: "partial",
          text,
          time: msg.time || 0
        });
        return;
      }
      if (type === "sentence") {
        sentenceWritten = true;
        const text = sanitizeText(msg.text || "");
        // 哪怕过滤后是空字符串，也应该告诉前端，让前端知道这部分处理完了，否则前端可能卡死或丢失状态
        const speakerLabel = mapSpeakerLabel(msg.speakerId);
        if (text) {
          const lineText = `[${speakerLabel}] ${text}\n`;
          try {
            fs.appendFile(transcriptFile, lineText, "utf8", (err) => {
              if (err) console.error("Error writing transcript file:", err);
              else console.log("ASR sentence transcript bytes:", Buffer.byteLength(lineText, "utf8"));
            });
          } catch (err) {
            console.error("Error writing transcript file:", err);
          }
          safeWsSend({
            type: "sentence",
            text,
            speakerId: speakerLabel
          });
        }
        return;
      }
      if (type === "completed") {
        flushFallbackText();
        safeWsSend({ type: "completed", file: transcriptFile });
        return;
      }
      if (type === "error") {
        const message = msg.message || "ASR bridge error";
        console.error("Python ASR error:", message);
        safeWsSend({ type: "error", message });
      }
    } catch (innerError) {
      console.error("ASR outputReader line processing error:", innerError);
    }
  });
  asrProcess.stderr.on("data", (chunk) => {
    const message = chunk.toString().trim();
    if (message) {
      console.error("Python ASR stderr:", message);
    }
  });
```

```
  asrProcess.on("close", (code) => {
    flushFallbackText();
    console.log("Python ASR process closed:", code);
    if (code !== 0) {
      safeWsSend({ type: "error", message: "ASR bridge process exited unexpectedly" });
    }
  });
} catch (e) {
  console.error("ASR Init Error:", e);
  safeWsSend({ type: "error", message: "ASR Init Error" });
}
let audioPacketCount = 0;
ws.on("message", (message, isBinary) => {
  if (asrProcess && asrProcess.stdin && !asrProcess.stdin.destroyed) {
    try {
      if (!isBinary) {
        return;
      }
      const audioBuffer = Buffer.isBuffer(message) ? message : Buffer.from(message);
      audioWriteStream.write(audioBuffer);
      audioPacketCount += 1;
      if (audioPacketCount <= 5) {
        let sum = 0;
        for (let i = 0; i + 1 < audioBuffer.length; i += 2) {
          const sample = audioBuffer.readInt16LE(i);
          sum += Math.abs(sample);
        }
        const avgAbs = audioBuffer.length > 1 ? Math.round(sum / (audioBuffer.length / 2)) : 0;
        console.log(`ASR audio packet #${audioPacketCount}, bytes=${audioBuffer.length}, avgAbs=${avgAbs}`);
      }
      asrProcess.stdin.write(audioBuffer);
    } catch (e) {
      console.error("Failed to write to ASR process stdin:", e);
    }
  }
});
ws.on("close", () => {
  console.log("Client disconnected from ASR WebSocket");
  if (asrProcess && asrProcess.stdin && !asrProcess.stdin.destroyed) {
    asrProcess.stdin.end();
  }
  if (!audioWriteStream.destroyed) {
    audioWriteStream.end(() => {
      finalizeAudioFile();
    });
  } else {
    finalizeAudioFile();
  }
  flushFallbackText();
  if (asrProcess && !asrProcess.killed) {
```

```
      setTimeout(() => {
        if (!asrProcess.killed) {
          asrProcess.kill("SIGTERM");
        }
      }, 1500);
    }
  });
 });
}
// ===== FILE END =====
// ===== FILE BEGIN: screen-catch\asr_bridge.py =====
import json
import os
import sys
import dashscope
from dashscope.audio.asr import Recognition, RecognitionCallback, RecognitionResult
def emit(event):
    sys.stdout.write(json.dumps(event, ensure_ascii=True) + "\n")
    sys.stdout.flush()
class Callback(RecognitionCallback):
    @staticmethod
    def _extract_speaker_id(sentence):
        speaker_id = sentence.get("speaker_id")
        if speaker_id is not None:
            return speaker_id
        speaker_id = sentence.get("speakerId")
        if speaker_id is not None:
            return speaker_id
        speaker_id = sentence.get("spk_id")
        if speaker_id is not None:
            return speaker_id
        words = sentence.get("words")
        if isinstance(words, list):
            counts = {}
            for w in words:
                if not isinstance(w, dict):
                    continue
                wid = w.get("speaker_id")
                if wid is None:
                    wid = w.get("speakerId")
                if wid is None:
                    continue
                key = str(wid)
                counts[key] = counts.get(key, 0) + 1
            if counts:
                return max(counts, key=counts.get)
        return None
    def on_open(self) -> None:
        emit({"type": "ready"})
    def on_complete(self) -> None:
```

```
        emit({"type": "completed"})
    def on_error(self, result) -> None:
        message = getattr(result, "message", str(result))
        request_id = getattr(result, "request_id", "")
        emit({"type": "error", "message": message, "request_id": request_id})
    def on_event(self, result: RecognitionResult) -> None:
        sentence = result.get_sentence()
        if not isinstance(sentence, dict):
            return
        text = sentence.get("text", "")
        if not text:
            return
        begin_time = sentence.get("begin_time", 0)
        end_time = sentence.get("end_time", 0)
        emit({"type": "partial", "text": text, "time": end_time})
        if RecognitionResult.is_sentence_end(sentence):
            speaker_id = self._extract_speaker_id(sentence)
            emit({"type": "sentence", "text": text, "speakerId": speaker_id, "time": end_time, "beginTime": begin_time})
def load_api_key():
    api_key = os.getenv("QWEN_API_KEY") or os.getenv("DASHSCOPE_API_KEY")
    if api_key:
        return api_key
    env_path = os.path.join(os.path.dirname(__file__), "..", ".env")
    if os.path.exists(env_path):
        with open(env_path, "r", encoding="utf-8") as f:
            for raw in f:
                line = raw.strip()
                if not line or line.startswith("#") or "=" not in line:
                    continue
                k, v = line.split("=", 1)
                k = k.strip()
                v = v.strip().strip('"').strip("'")
                if k == "QWEN_API_KEY" and v:
                    return v
                if k == "DASHSCOPE_API_KEY" and v:
                    return v
    return None
def main():
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    api_key = load_api_key()
    if not api_key:
        emit({"type": "error", "message": "Missing QWEN_API_KEY or DASHSCOPE_API_KEY"})
        return 1
    dashscope.api_key = api_key
    model = os.getenv("ASR_MODEL", "paraformer-realtime-v2")
    fmt = os.getenv("ASR_FORMAT", "pcm")
    sample_rate = int(os.getenv("ASR_SAMPLE_RATE", "16000"))
    recognition = Recognition(
        model=model,
```

```
        format=fmt,
        sample_rate=sample_rate,
        language_hints=["zh", "en"],
        diarization_enabled=True,
        semantic_punctuation_enabled=False,
        max_sentence_silence=800,
        punctuation_prediction_enabled=True,
        inverse_text_normalization_enabled=True,
        callback=Callback(),
    )
    recognition.start()
    try:
        while True:
            data = sys.stdin.buffer.read(3200)
            if not data:
                break
            recognition.send_audio_frame(data)
    except Exception as e:
        emit({"type": "error", "message": f"ASR Stream Error: {str(e)}"})
    finally:
        try:
            recognition.stop()
        except Exception:
            pass
    return 0
if __name__ == "__main__":
    try:
        raise SystemExit(main())
    except Exception as e:
        emit({"type": "error", "message": str(e)})
        raise
// ===== FILE END =====
// ===== FILE BEGIN: screen-catch\asr_finalize.py =====
import json
import os
import re
import sys
import urllib.error
import urllib.parse
import urllib.request
import dashscope
from dashscope.audio.asr import Transcription
def load_api_key():
    api_key = os.getenv("QWEN_API_KEY") or os.getenv("DASHSCOPE_API_KEY")
    if api_key:
        return api_key
    env_path = os.path.join(os.path.dirname(__file__), "..", ".env")
    if os.path.exists(env_path):
        content = open(env_path, "r", encoding="utf-8").read()
        m = re.search(r"^QWEN_API_KEY=(.*)$", content, flags=re.M)
```

```
        if m:
            return m.group(1).strip().strip('"').strip("'")
        m2 = re.search(r"^DASHSCOPE_API_KEY=(.*)$", content, flags=re.M)
        if m2:
            return m2.group(1).strip().strip('"').strip("'")
    return None
def extract_speaker_id(sentence):
    if not isinstance(sentence, dict):
        return None
    for k in ("speaker_id", "speakerId", "spk_id"):
        v = sentence.get(k)
        if v is not None and v != "":
            return v
    words = sentence.get("words")
    if isinstance(words, list):
        counts = {}
        for w in words:
            if not isinstance(w, dict):
                continue
            wid = w.get("speaker_id")
            if wid is None:
                wid = w.get("speakerId")
            if wid is None or wid == "":
                continue
            key = str(wid)
            counts[key] = counts.get(key, 0) + 1
        if counts:
            return max(counts, key=counts.get)
    return None
def is_http_url(value):
    return isinstance(value, str) and value.startswith(("http://", "https://"))
def get_data_dir():
    return os.path.abspath(os.path.join(os.path.dirname(__file__), "data"))
def build_public_audio_url(local_path):
    public_base = (os.getenv("ASR_AUDIO_PUBLIC_BASE_URL") or "").strip().rstrip("/")
    if not public_base:
        return None
    data_dir = get_data_dir()
    abs_path = os.path.abspath(local_path)
    try:
        rel_path = os.path.relpath(abs_path, data_dir)
    except ValueError:
        return None
    if rel_path.startswith(".."):
        return None
    rel_url = urllib.parse.quote(rel_path.replace("\\", "/"))
    return f"{public_base}/api/asr/files/{rel_url}"
def normalize_audio_source(audio_source):
    if is_http_url(audio_source):
        return audio_source
```

```
    if not os.path.exists(audio_source):
        return None
    return build_public_audio_url(audio_source)
def fetch_json(url):
    with urllib.request.urlopen(url, timeout=60) as response:
        raw = response.read()
    return json.loads(raw.decode("utf-8"))
def extract_transcription_url(payload):
    if not isinstance(payload, dict):
        return None
    output = payload.get("output") or {}
    result = output.get("result")
    if isinstance(result, dict) and result.get("transcription_url"):
        return result.get("transcription_url")
    results = output.get("results")
    if isinstance(results, list):
        for item in results:
            if isinstance(item, dict) and item.get("transcription_url"):
                return item.get("transcription_url")
    return None
def extract_sentences_from_payload(payload):
    if not isinstance(payload, dict):
        return []
    transcripts = payload.get("transcripts")
    if not isinstance(transcripts, list):
        return []
    sentences = []
    for transcript in transcripts:
        if not isinstance(transcript, dict):
            continue
        for s in transcript.get("sentences", []) or []:
            if not isinstance(s, dict):
                continue
            text = s.get("text", "")
            if not text:
                continue
            sentences.append(
                {
                    "text": text,
                    "speakerId": extract_speaker_id(s),
                    "beginTime": s.get("begin_time", 0),
                    "endTime": s.get("end_time", 0),
                }
            )
    return sentences
def response_to_dict(response):
    if isinstance(response, dict):
        return {k: response_to_dict(v) for k, v in response.items()}
    if isinstance(response, list):
        return [response_to_dict(v) for v in response]
```

```
    if isinstance(response, (str, int, float, bool)) or response is None:
        return response
    if hasattr(response, "__dict__"):
        return {
            k: response_to_dict(v)
            for k, v in response.__dict__.items()
            if not k.startswith("_")
        }
    return str(response)
def main():
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    if len(sys.argv) < 2:
        print(json.dumps({"ok": False, "error": "missing_wav_path"}, ensure_ascii=True))
        return 1
    audio_source = sys.argv[1]
    if not is_http_url(audio_source) and not os.path.exists(audio_source):
        print(json.dumps({"ok": False, "error": "audio_source_not_found"}, ensure_ascii=True))
        return 1
    api_key = load_api_key()
    if not api_key:
        print(json.dumps({"ok": False, "error": "missing_api_key"}, ensure_ascii=True))
        return 1
    dashscope.api_key = api_key
    model = os.getenv("ASR_FINALIZE_MODEL", "paraformer-v2")
    public_audio_url = normalize_audio_source(audio_source)
    if not public_audio_url:
        print(
            json.dumps(
                {
                    "ok": False,
                    "error": "missing_public_audio_url",
                    "message": "Paraformer file transcription requires a public audio URL. Set ASR_AUDIO_PUBLIC_BASE_
                    "model": model,
                },
                ensure_ascii=True,
            )
        )
        return 1
    try:
        task = Transcription.async_call(
            model=model,
            file_urls=[public_audio_url],
            diarization_enabled=True,
            timestamp_alignment_enabled=True,
            language_hints=["zh", "en"],
        )
        result = Transcription.wait(task)
        payload = response_to_dict(result)
        try:
```

```
            status_code = int(payload.get("status_code", 0))
        except Exception:
            status_code = 0
        if status_code != 200:
            print(
                json.dumps(
                    {
                        "ok": False,
                        "error": str(payload.get("message") or payload.get("code") or "finalize_asr_failed"),
                        "model": model,
                        "audioUrl": public_audio_url,
                        "payload": payload,
                    },
                    ensure_ascii=True,
                )
            )
            return 1
        transcription_url = extract_transcription_url(payload)
        if not transcription_url:
            print(
                json.dumps(
                    {
                        "ok": False,
                        "error": "missing_transcription_url",
                        "model": model,
                        "audioUrl": public_audio_url,
                        "payload": payload,
                    },
                    ensure_ascii=True,
                )
            )
            return 1
        result_payload = fetch_json(transcription_url)
        sentences = extract_sentences_from_payload(result_payload)
        print(
            json.dumps(
                {
                    "ok": True,
                    "model": model,
                    "audioUrl": public_audio_url,
                    "transcriptionUrl": transcription_url,
                    "sentences": sentences,
                },
                ensure_ascii=True,
            )
        )
        return 0
    except urllib.error.URLError as exc:
        print(
            json.dumps(
```

```
                {
                    "ok": False,
                    "error": "fetch_transcription_result_failed",
                    "message": str(exc),
                    "model": model,
                    "audioUrl": public_audio_url,
                },
                ensure_ascii=True,
            )
        )
        return 1
    except Exception as exc:
        print(
            json.dumps(
                {
                    "ok": False,
                    "error": "finalize_asr_failed",
                    "message": str(exc),
                    "model": model,
                    "audioUrl": public_audio_url,
                },
                ensure_ascii=True,
            )
        )
        return 1
if __name__ == "__main__":
    raise SystemExit(main())
// ===== FILE END =====
// ===== FILE BEGIN: server\src\analyze.js =====
import { extractLikelyJsonObject, safeJsonParse } from "./json.js";
import {
  buildAnalyzeMessages,
  buildCorrectTranscriptMessages,
  buildFixJsonMessages,
  buildFollowUpQuestionsMessages,
  buildParticipantsMessages,
  buildTopicsReportMessages,
  OUTPUT_SCHEMA
} from "./prompts.js";
import { createQwenClient, getQwenConfig } from "./qwenClient.js";
import { nowIso, writeNdjson } from "./ndjson.js";
function validateShape(obj) {
  if (!obj || typeof obj !== "object") return { ok: false, error: "not_object" };
  for (const k of Object.keys(OUTPUT_SCHEMA)) {
    if (typeof obj[k] !== "string") return { ok: false, error: `missing_or_not_string:${k}` };
  }
  return { ok: true };
}
async function completeJson(messages) {
  const client = createQwenClient();
```

```
  const { model, enableThinking, maxTokens } = getQwenConfig();
  const resp = await client.chat.completions.create({
    model,
    messages,
    temperature: 0.2,
    max_tokens: maxTokens,
    extra_body: { enable_thinking: enableThinking }
  });
  return resp.choices?.[0]?.message?.content ?? "";
}
export async function analyzeTranscript(transcriptText) {
  const raw = await completeJson(buildAnalyzeMessages(transcriptText));
  const extracted = extractLikelyJsonObject(raw) ?? raw;
  const parsed1 = safeJsonParse(extracted);
  if (parsed1.ok) {
    const shape = validateShape(parsed1.value);
    if (shape.ok) return { ok: true, value: parsed1.value, raw };
  }
  const fixedRaw = await completeJson(buildFixJsonMessages(raw));
  const fixedExtracted = extractLikelyJsonObject(fixedRaw) ?? fixedRaw;
  const parsed2 = safeJsonParse(fixedExtracted);
  if (!parsed2.ok) return { ok: false, error: "json_parse_failed", raw, fixedRaw };
  const shape2 = validateShape(parsed2.value);
  if (!shape2.ok) return { ok: false, error: shape2.error, raw, fixedRaw };
  return { ok: true, value: parsed2.value, raw };
}
async function streamMarkdownSection({ section, messages, res }) {
  const client = createQwenClient();
  const { model, maxTokens } = getQwenConfig();
  const startedAt = Date.now();
  let firstTokenAt = null;
  writeNdjson(res, { type: "log", ts: nowIso(), message: `开始生成：${section}` });
  const stream = await client.chat.completions.create({
    model,
    messages,
    temperature: 0.2,
    max_tokens: maxTokens,
    stream: true,
    extra_body: { enable_thinking: false }
  });
  let content = "";
  for await (const part of stream) {
    const delta = part.choices?.[0]?.delta?.content ?? "";
    if (!delta) continue;
    if (firstTokenAt === null) {
      firstTokenAt = Date.now();
      writeNdjson(res, {
        type: "log",
        ts: nowIso(),
        message: `${section} 首token耗时：${firstTokenAt - startedAt}ms`
```

```
      });
    }
    content += delta;
    writeNdjson(res, { type: "delta", ts: nowIso(), section, delta });
  }
  const endedAt = Date.now();
  writeNdjson(res, { type: "section_done", ts: nowIso(), section });
  writeNdjson(res, {
    type: "log",
    ts: nowIso(),
    message: `${section} 生成耗时：${endedAt - startedAt}ms`
  });
  return {
    content: content.trim(),
    startedAt,
    firstTokenAt,
    endedAt
  };
}
export async function analyzeCaseContent(images, transcriptText, contextTranscriptText, previousAnalysis) {
  const client = createQwenClient();
  const schemaStr = JSON.stringify(OUTPUT_SCHEMA, null, 2);
  const contentArray = [];
  let prevStr = previousAnalysis;
  if (typeof previousAnalysis === "object" && previousAnalysis !== null) {
    prevStr = JSON.stringify({
      participantsAndViewpointsMd: previousAnalysis.participantsAndViewpointsMd,
      topicsReportMd: previousAnalysis.topicsReportMd,
      followUpQuestionsMd: previousAnalysis.followUpQuestionsMd,
      glossaryMd: previousAnalysis.glossaryMd
    }, null, 2);
  }
  // 提前在这里声明并处理好截断版本，保证 Agent 1 和 Agent 2 都能访问到
  let trimmedPrevStr = prevStr || "";
  if (trimmedPrevStr.length > 20000) {
    trimmedPrevStr = trimmedPrevStr.slice(-20000) + "\n...（因长度限制已截断部分早期历史总结）";
  }
  let trimmedContext = contextTranscriptText || '';
  if (trimmedContext.length > 8000) {
    trimmedContext = "...\n" + trimmedContext.slice(-8000);
  }
  let trimmedTranscript = transcriptText || '';
  if (trimmedTranscript.length > 10000) {
    trimmedTranscript = "...\n" + trimmedTranscript.slice(-10000);
  }
  if (prevStr) {
    contentArray.push({ type: "text", text: `这是前次生成的分析总结结果（不包含历史转写原文）：\n\n${trimmedPrevS
    contentArray.push({ type: "text", text: `这是为了保持连贯性提供的上一段转写上下文：\n${trimmedContext || '（无上
    contentArray.push({ type: "text", text: `以下是最新增加的会议语音转写记录：\n${trimmedTranscript || '（无新增转写
    contentArray.push({ type: "text", text: `以下是最新增加的会议截屏（附带截屏时间）：` });
```

```
} else {
  contentArray.push({ type: "text", text: `这是一场会议的语音转写记录：\n${trimmedTranscript}\n\n以下是会议过程中
}
if (images.length === 0) {
  contentArray.push({ type: "text", text: `\n（无新增截屏）` });
} else {
  for (const img of images) {
    contentArray.push({ type: "text", text: `\n[时间: ${img.time}]` });
    contentArray.push({ type: "image_url", image_url: { url: img.imageUrl } });
  }
}
const outputReq = [
  `\n\n请结合多张会议截屏（注意时间顺序与内容变化）以及会议语音转写记录，深入分析并输出技术报告。`,
  `输出要求：`,
  `- 只输出JSON，不要输出任何额外文字。`,
  `- JSON必须严格符合以下键结构，值全部为Markdown字符串：`,
  schemaStr,
  `- 五个字段分别对应前端独立板块展示，内容不要互相混杂：`,
  ` correctedTranscriptMd: (此字段已在前端暂时屏蔽，你可以返回空字符串或简略信息以节省时间，因为不再展示)`,
  ` participantsAndViewpointsMd: 参与者与观点（每人单独小节，包含其对各议题的立场/理解与变化）。【重要】：这
  ` topicsReportMd: 议题报告（按时间顺序逐议题展开，必须包含：初始现状、讨论过程关键点、最终共识、前后差异、
  ` followUpQuestionsMd: 追问清单（按议题组织，包含：提问对象、原始问题、提问原因）。【极其重要】：你必须按
  ` glossaryMd: 术语表（按字母或拼音或出现顺序均可，每项给出解释与在本会议中的语境）。`
];
if (prevStr) {
  outputReq.push(`\n重要约束：
1. 请在"前次生成的分析总结结果"基础上，结合新增内容进行全局更新。
2. 【极端重要】对于 \`correctedTranscriptMd\`，(前端已屏蔽，直接返回空字符串即可)。
3. 对于其他 4 个分析字段（参与者、议题、追问、术语表），请综合前次结果与新增内容，输出一份【包含之前所有内容的
   【极其重要警告】：你绝对、绝对不能为了"简明扼要"而删掉前次结果中已经列出的历史人员、历史观点、历史议题或历
4. 如果除了转写外，其他分析字段无明显变化，保持前次分析的内容原样输出即可。`);
}
contentArray.push({ type: "text", text: outputReq.join("\n") });
const { model, maxTokens } = getQwenConfig();
// 如果没有新增截屏，退回到使用配置中的普通文本大模型，以防 qwen-vl 报错无图
const targetModel = images.length === 0 ? model : "qwen-vl-max";
const messages = [
  {
    role: "system",
    content: "你是一个专业的会议分析助手。你需要结合用户提供的屏幕截图和会议语音转写记录，输出指定格式的 JSON
  },
  {
    role: "user",
    content: targetModel === "qwen-vl-max" ? contentArray : contentArray.map(c => c.text).join("\n")
  }
];
const resp = await client.chat.completions.create({
  model: targetModel,
  messages,
  temperature: 0.2,
```

```
    max_tokens: maxTokens || 8192
  });
    const raw = resp.choices?.[0]?.message?.content ?? "";
    let extracted = extractLikelyJsonObject(raw) ?? raw;
    let parsed = safeJsonParse(extracted);
    if (!parsed.ok) {
      // 增加一行正则替换：如果模型在 JSON 中输出了不合法的转义换行符或结尾多余逗号，尝试先用模型自我修复
      const fixedRaw = await completeJson(buildFixJsonMessages(raw));
      extracted = extractLikelyJsonObject(fixedRaw) ?? fixedRaw;
      parsed = safeJsonParse(extracted);
    }
    if (parsed.ok) {
      // ===== 跳过 Agent 2 调用，直接返回，以节省时间 =====
      return parsed.value;
    }
    // 即使解析失败，我们也不直接报错中断整个会议，而是返回一个部分成功的默认结构，或者打平抛出
    console.error("JSON Parse failed after fix:", parsed.error, "RAW:", raw);
    // 如果之前有数据，退回使用之前的数据
    if (typeof previousAnalysis === "object" && previousAnalysis !== null) {
      return previousAnalysis;
    }
    // 否则抛出明确的错误，告知前端发生了 JSON 解析失败
    throw new Error(`Failed to generate valid JSON analysis. Parse Error: ${parsed.error?.message || "unknown"}`);
  }
export async function analyzeTranscriptStream(transcriptText, res) {
  const runStartedAt = Date.now();
  let globalFirstTokenAt = null;
  writeNdjson(res, { type: "log", ts: nowIso(), message: "读取转写文本完成" });
  // 暂时屏蔽由大模型生成修正后的转写，直接使用原始的实时转写内容
  // 并且将后续的依赖全部替换为原始的实时转写 (transcriptText)，保证其他功能正常运行
  const correctedTranscriptMd = transcriptText;
  if (globalFirstTokenAt === null) {
    globalFirstTokenAt = Date.now();
  }
  const participantsAndViewpointsMd = (
    await streamMarkdownSection({
      section: "participantsAndViewpointsMd",
      messages: buildParticipantsMessages(transcriptText),
      res
    })
  ).content;
  const topicsReportMd = (
    await streamMarkdownSection({
      section: "topicsReportMd",
      messages: buildTopicsReportMessages(transcriptText),
      res
    })
  ).content;
  const followUpQuestionsMd = (
    await streamMarkdownSection({
```

```
      section: "followUpQuestionsMd",
      messages: buildFollowUpQuestionsMessages(transcriptText),
      res
    })
  ).content;
  const glossaryMd = "";
  const runEndedAt = Date.now();
  writeNdjson(res, { type: "log", ts: nowIso(), message: `总耗时：${runEndedAt - runStartedAt}ms` });
  writeNdjson(res, {
    type: "done",
    ts: nowIso(),
    data: {
      correctedTranscriptMd,
      participantsAndViewpointsMd,
      topicsReportMd,
      followUpQuestionsMd,
      glossaryMd
    }
  });
}
// ===== FILE END =====
// ===== FILE BEGIN: server\src\chatStore.js =====
import fs from "node:fs";
import path from "node:path";
function getChatUsersRoot() {
  return path.resolve(process.cwd(), "..", "screen-catch", "data", "chat-users");
}
function getChatUsersDir() {
  return path.join(getChatUsersRoot(), "users");
}
function getUserDir(userId) {
  return path.join(getChatUsersDir(), userId);
}
function getUserMetaPath(userId) {
  return path.join(getUserDir(userId), "meta.json");
}
function getConversationDir(userId) {
  return path.join(getUserDir(userId), "conversations");
}
function getConversationPath(userId, slug) {
  return path.join(getConversationDir(userId), `${slug}.json`);
}
function getSeqPath() {
  return path.join(getChatUsersRoot(), "user-id-seq.json");
}
function ensureDir(dirPath) {
  if (!fs.existsSync(dirPath)) {
    fs.mkdirSync(dirPath, { recursive: true });
  }
}
```

```
function ensureChatStoreDirs() {
  ensureDir(getChatUsersRoot());
  ensureDir(getChatUsersDir());
}
function readJson(filePath, fallback) {
  try {
    if (!fs.existsSync(filePath)) return fallback;
    return JSON.parse(fs.readFileSync(filePath, "utf8"));
  } catch {
    return fallback;
  }
}
function writeJson(filePath, value) {
  fs.writeFileSync(filePath, JSON.stringify(value, null, 2), "utf8");
}
function formatYYMMDD(date = new Date()) {
  const year = String(date.getFullYear()).slice(-2);
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");
  return `${year}${month}${day}`;
}
function randomSuffix() {
  return String(Math.floor(Math.random() * 10000)).padStart(4, "0");
}
function sanitizeStoredMessages(messages = []) {
  if (!Array.isArray(messages)) return [];
  return messages
    .filter((item) => item && typeof item.content === "string")
    .map((item) => {
      const role = item.role === "user" || item.role === "spectator" ? item.role : "assistant";
      return {
        id: typeof item.id === "string" ? item.id : "",
        role,
        content: item.content,
        label: typeof item.label === "string" ? item.label : ""
      };
    });
}
function summarizeConversation(conversation) {
  const messages = Array.isArray(conversation?.messages) ? conversation.messages : [];
  const lastMessage = messages[messages.length - 1];
  return {
    slug: conversation.slug || "",
    profileName: conversation.profileName || "",
    messageCount: messages.length,
    updatedAt: conversation.updatedAt || conversation.createdAt || "",
    lastMessagePreview: lastMessage?.content ? lastMessage.content.slice(0, 120) : ""
  };
}
export function isValidChatUserId(userId) {
```

```
  return /^\d{10}$/.test(String(userId || "").trim());
}
export function allocateChatUserId() {
  ensureChatStoreDirs();
  const seqPath = getSeqPath();
  const seqState = readJson(seqPath, {});
  const prefix = formatYYMMDD();
  const usedSuffixes = new Set(Array.isArray(seqState[prefix]?.used_suffixes) ? seqState[prefix].used_suffixes : []);
  let suffix = "";
  for (let i = 0; i < 200; i += 1) {
    const candidate = randomSuffix();
    if (!usedSuffixes.has(candidate)) {
      suffix = candidate;
      break;
    }
  }
  if (!suffix) {
    for (let i = 0; i < 10000; i += 1) {
      const candidate = String(i).padStart(4, "0");
      if (!usedSuffixes.has(candidate)) {
        suffix = candidate;
        break;
      }
    }
  }
  if (!suffix) {
    throw new Error("user_id_exhausted");
  }
  usedSuffixes.add(suffix);
  seqState[prefix] = { used_suffixes: Array.from(usedSuffixes).sort() };
  writeJson(seqPath, seqState);
  const userId = `${prefix}${suffix}`;
  return ensureChatUser(userId).data;
}
export function ensureChatUser(userId) {
  const normalizedUserId = String(userId || "").trim();
  if (!isValidChatUserId(normalizedUserId)) {
    return { ok: false, error: "invalid_user_id" };
  }
  ensureChatStoreDirs();
  const userDir = getUserDir(normalizedUserId);
  const conversationsDir = getConversationDir(normalizedUserId);
  ensureDir(userDir);
  ensureDir(conversationsDir);
  const metaPath = getUserMetaPath(normalizedUserId);
  const existing = readJson(metaPath, null);
  const now = new Date().toISOString();
  const meta = existing || {
    userId: normalizedUserId,
    createdAt: now,
```

```
    updatedAt: now
  };
  meta.updatedAt = now;
  writeJson(metaPath, meta);
  return { ok: true, data: meta };
}
export function getChatUser(userId) {
  const normalizedUserId = String(userId || "").trim();
  if (!isValidChatUserId(normalizedUserId)) {
    return null;
  }
  return readJson(getUserMetaPath(normalizedUserId), null);
}
export function saveConversationState(userId, slug, { profileName = "", messages = [], autopilotReport = "" } = {}
  const ensured = ensureChatUser(userId);
  if (!ensured.ok) return ensured;
  const normalizedSlug = String(slug || "").trim();
  if (!normalizedSlug) {
    return { ok: false, error: "missing_slug" };
  }
  const filePath = getConversationPath(ensured.data.userId, normalizedSlug);
  const existing = readJson(filePath, null);
  const now = new Date().toISOString();
  const conversation = {
    userId: ensured.data.userId,
    slug: normalizedSlug,
    profileName: String(profileName || existing?.profileName || "").trim(),
    createdAt: existing?.createdAt || now,
    updatedAt: now,
    messages: sanitizeStoredMessages(messages),
    autopilotReport: typeof autopilotReport === "string" ? autopilotReport : ""
  };
  writeJson(filePath, conversation);
  const meta = {
    ...ensured.data,
    updatedAt: now
  };
  writeJson(getUserMetaPath(ensured.data.userId), meta);
  return { ok: true, data: conversation };
}
export function loadConversationState(userId, slug) {
  const normalizedUserId = String(userId || "").trim();
  const normalizedSlug = String(slug || "").trim();
  if (!isValidChatUserId(normalizedUserId) || !normalizedSlug) {
    return { ok: false, error: "invalid_user_id_or_slug" };
  }
  const conversation = readJson(getConversationPath(normalizedUserId, normalizedSlug), null);
  if (!conversation) {
    return {
      ok: true,
```

```
      data: {
        userId: normalizedUserId,
        slug: normalizedSlug,
        profileName: "",
        createdAt: "",
        updatedAt: "",
        messages: [],
        autopilotReport: ""
      }
    };
  }
  return {
    ok: true,
    data: {
      ...conversation,
      messages: sanitizeStoredMessages(conversation.messages),
      autopilotReport: typeof conversation.autopilotReport === "string" ? conversation.autopilotReport : ""
    }
  };
}
export function listUserConversations(userId) {
  const normalizedUserId = String(userId || "").trim();
  if (!isValidChatUserId(normalizedUserId)) {
    return [];
  }
  const conversationsDir = getConversationDir(normalizedUserId);
  if (!fs.existsSync(conversationsDir)) return [];
  const items = [];
  for (const entry of fs.readdirSync(conversationsDir, { withFileTypes: true })) {
    if (!entry.isFile() || !entry.name.endsWith(".json")) continue;
    const conversation = readJson(path.join(conversationsDir, entry.name), null);
    if (conversation) {
      items.push(summarizeConversation(conversation));
    }
  }
  return items.sort((a, b) => String(b.updatedAt || "").localeCompare(String(a.updatedAt || "")));
}
export function listChatUsers() {
  ensureChatStoreDirs();
  const users = [];
  for (const entry of fs.readdirSync(getChatUsersDir(), { withFileTypes: true })) {
    if (!entry.isDirectory()) continue;
    const meta = readJson(getUserMetaPath(entry.name), null);
    if (!meta) continue;
    const conversations = listUserConversations(entry.name);
    users.push({
      userId: meta.userId || entry.name,
      createdAt: meta.createdAt || "",
      updatedAt: meta.updatedAt || "",
      conversationCount: conversations.length,
```

```
      conversations
    });
  }
  return users.sort((a, b) => String(b.updatedAt || "").localeCompare(String(a.updatedAt || "")));
}
export function getChatUserDetail(userId) {
  const meta = getChatUser(userId);
  if (!meta) return null;
  const conversations = [];
  const conversationsDir = getConversationDir(meta.userId);
  if (fs.existsSync(conversationsDir)) {
    for (const entry of fs.readdirSync(conversationsDir, { withFileTypes: true })) {
      if (!entry.isFile() || !entry.name.endsWith(".json")) continue;
      const conversation = readJson(path.join(conversationsDir, entry.name), null);
      if (!conversation) continue;
      conversations.push({
        ...conversation,
        messages: sanitizeStoredMessages(conversation.messages)
      });
    }
  }
  conversations.sort((a, b) => String(b.updatedAt || "").localeCompare(String(a.updatedAt || "")));
  return {
    userId: meta.userId,
    createdAt: meta.createdAt || "",
    updatedAt: meta.updatedAt || "",
    conversationCount: conversations.length,
    conversations
  };
}
// ===== FILE END =====
// ===== FILE BEGIN: server\src\env.js =====
import dotenv from "dotenv";
import path from "node:path";
export function loadEnv() {
  const candidates = [
    path.resolve(process.cwd(), ".env"),
    path.resolve(process.cwd(), "..", ".env")
  ];
  for (const p of candidates) {
    const result = dotenv.config({ path: p, override: true });
    if (!result.error) return;
  }
}
export function getRequiredEnv(name) {
  const v = process.env[name];
  if (!v) throw new Error(`Missing env: ${name}`);
  return v;
}
// ===== FILE END =====
```

```
// ===== FILE BEGIN: server\src\index.js =====
import cors from "cors";
import express from "express";
import multer from "multer";
import path from "node:path";
import fs from "node:fs";
import http from "node:http";
import { loadEnv } from "./env.js";
import { analyzeTranscript, analyzeTranscriptStream, analyzeCaseContent } from "./analyze.js";
import { createQwenClient, getQwenConfig } from "./qwenClient.js";
import { setupScreenCatchRoutes } from "../../screen-catch/api.js";
import {
  buildProfileForSpeaker,
  createManualProfile,
  mergeProfileForSpeaker,
  upsertWorkProfileFromAnalysis,
  loadProfileBySlug,
  listProfiles,
  updateProfileNameBySlug,
  updateSpeakerName,
  guessSpeakers,
  extractSpeakersFromTranscript,
  deleteProfile
} from "./profileBuilder.js";
import {
  allocateChatUserId,
  ensureChatUser,
  getChatUser,
  saveConversationState,
  loadConversationState,
  listChatUsers,
  getChatUserDetail,
  isValidChatUserId
} from "./chatStore.js";
import { getOrBuildUserProfile, loadStoredUserProfile } from "./userProfile.js";
import { buildChatSystemPrompt } from "./profilePrompts.js";
import { extractLikelyJsonObject, safeJsonParse } from "./json.js";
// 配置日志同时输出到文件
const logPath = path.resolve(process.cwd(), "../../screen-catch", "data", "server.log");
const logDir = path.dirname(logPath);
if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir, { recursive: true });
}
const originalConsoleLog = console.log;
const originalConsoleError = console.error;
console.log = (...args) => {
  const logLine = `[${new Date().toISOString()}] ${args.map(arg =>
    typeof arg === "object" ? JSON.stringify(arg) : String(arg)
  ).join(" ")}\n`;
  fs.appendFileSync(logPath, logLine, "utf8");
```

```
  originalConsoleLog.apply(console, args);
};
console.error = (...args) => {
  const logLine = `[${new Date().toISOString()}] ERROR: ${args.map(arg =>
    typeof arg === "object" ? JSON.stringify(arg) : String(arg)
  ).join(" ")}\n`;
  fs.appendFileSync(logPath, logLine, "utf8");
  originalConsoleError.apply(console, args);
};
loadEnv();
const app = express();
const server = http.createServer(app);
app.use(cors());
app.use(express.json({ limit: "50mb" }));
setupScreenCatchRoutes(app, server);
const staticDir = path.resolve(process.cwd(), "..", "web-static");
app.use(express.static(staticDir));
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 }
});
function normalizeChatMessages(messages = []) {
  if (!Array.isArray(messages)) return [];
  return messages
    .filter((item) => item && (item.role === "user" || item.role === "assistant") && typeof item.content === "strin
    .map((item) => ({ role: item.role, content: item.content }));
}
function normalizeConversationMessages(messages = []) {
  if (!Array.isArray(messages)) return [];
  return messages
    .filter((item) => item && typeof item.content === "string")
    .map((item) => ({
      id: typeof item.id === "string" ? item.id : "",
      role: item.role === "user" || item.role === "spectator" ? item.role : "assistant",
      content: item.content,
      label: typeof item.label === "string" ? item.label : ""
    }));
}
function formatChatHistory(messages = []) {
  if (!messages.length) return "（暂无对话历史）";
  return messages
    .map((item) => `${item.role === "user" ? "用户" : "模拟同事"}：${item.content}`)
    .join("\n\n");
}
function formatAutopilotTranscript(items = []) {
  if (!Array.isArray(items) || items.length === 0) return "（暂无自动讨论记录）";
  return items
    .filter((item) => item && typeof item.content === "string")
    .map((item) => `${item.speaker === "spectator" ? "旁观者" : "模拟同事"}：${item.content}`)
    .join("\n\n");
```

```
}
function parseJsonContent(raw) {
  if (!raw || typeof raw !== "string") return { ok: false, error: "empty_content" };
  const extracted = extractLikelyJsonObject(raw) ?? raw;
  const parsed = safeJsonParse(extracted);
  if (parsed.ok) return parsed;
  const cleaned = raw.replace(/```json\s*/g, "").replace(/```\s*/g, "").trim();
  return safeJsonParse(extractLikelyJsonObject(cleaned) ?? cleaned);
}
async function completeJsonObject(messages, { temperature = 0.2, maxTokens } = {}) {
  const qwenClient = createQwenClient();
  const { model, maxTokens: defaultMaxTokens } = getQwenConfig();
  const resp = await qwenClient.chat.completions.create({
    model,
    messages,
    temperature,
    max_tokens: maxTokens || Math.min(defaultMaxTokens, 2048),
    response_format: { type: "json_object" },
    extra_body: { enable_thinking: false }
  });
  return resp.choices?.[0]?.message?.content ?? "";
}
async function streamCompletionAsText(res, messages, { temperature = 0.2, maxTokens } = {}) {
  const qwenClient = createQwenClient();
  const { model, maxTokens: defaultMaxTokens } = getQwenConfig();
  const stream = await qwenClient.chat.completions.create({
    model,
    messages,
    max_tokens: maxTokens || defaultMaxTokens,
    stream: true,
    temperature,
    extra_body: { enable_thinking: false }
  });
  for await (const chunk of stream) {
    const delta = chunk.choices?.[0]?.delta?.content;
    if (delta) {
      res.write(delta);
    }
  }
}
function buildSpectatorSuggestionMessages({ profile, messages, draftMessage, pendingSuggestions, manual }) {
  const chatSystem = buildChatSystemPrompt(profile);
  const historyText = formatChatHistory(messages);
  const suggestionText = Array.isArray(pendingSuggestions) && pendingSuggestions.length > 0
    ? pendingSuggestions.map((item, idx) => `- ${idx + 1}. ${item.topic || "未命名建议"}：${item.assistantPrompt || i
    : "（当前没有待处理建议）";
  const draftText = typeof draftMessage === "string" && draftMessage.trim() ? draftMessage.trim() : "（当前输入
  const system = [
    "你是一个旁观者智能体（Spectator）。",
    "你的职责不是直接替用户回答，而是在旁边观察“用户与模拟同事”的对话，并在合适时机提出一个高价值建议。",
```

```
            <div class="hint-block">{{ item.assistantPrompt }}</div>
            <div class="card-meta" v-if="item.reasoning" style="margin-top:6px;">理由：{{ item.reasoning }}</di
            <div class="suggestion-actions">
              <button class="btn secondary" @click="applySuggestion(item)">填入输入框</button>
              <button class="btn secondary" @click="sendSuggestionNow(item)">直接发送</button>
              <button class="btn secondary" @click="dismissSuggestion(item.id)">忽略</button>
            </div>
          </div>
        </div>
      </div>
    </div>
    <div class="panel-section" v-if="autopilotReport">
      <div class="section-header"><div class="section-title">讨论总结</div></div>
      <div class="report-box"><div class="message-content md" v-html="renderMarkdown(autopilotReport)"
    </div>
  </div>
 </div>
</div>
<script>
 const { createApp, ref, computed, nextTick, watch } = Vue;
 const ACTIVE_CHAT_USER_KEY = "meet-helper.active.chat.user";
 createApp({
  setup() {
    const initialSlug = new URLSearchParams(window.location.search).get("slug");
    const profiles = ref([]);
    const selectedSlug = ref(null);
    const messages = ref([]);
    const inputText = ref("");
    const isTyping = ref(false);
    const messagesRef = ref(null);
    const inputRef = ref(null);
    const pendingSuggestions = ref([]);
    const spectatorLoading = ref(false);
    const spectatorStatus = ref("Spectator 会在合适时机主动提示。");
    const autopilotRunning = ref(false);
    const autopilotMode = ref("normal");
    const autopilotTranscript = ref([]);
    const autopilotReport = ref("");
    const autopilotStatus = ref("尚未开始自动讨论");
    const autopilotRound = ref(0);
    const autopilotStopRequested = ref(false);
    const chatUserId = ref("");
    const loginInput = ref("");
    const loginLoading = ref(false);
    const loginStatus = ref("未登录编号用户");
    const conversationStatus = ref("请选择编号并开始对话。");
    const loadingConversation = ref(false);
    let suggestionTimer = null;
    let autopilotToken = 0;
    const selectedProfile = computed(() => profiles.value.find((p) => p.slug === selectedSlug.value) || null);
```

```
const canInteract = computed(() => Boolean(chatUserId.value && selectedSlug.value));
function newId(prefix) {
  return `${prefix}-${Date.now()}-${Math.random().toString(16).slice(2, 8)}`;
}
function renderMarkdown(text) {
  if (!text) return "";
  return DOMPurify.sanitize(marked.parse(text));
}
function messageClass(msg) {
  if (msg.role === "spectator") return "spectator";
  return msg.role === "user" ? "user" : "assistant";
}
function messageAvatar(msg) {
  if (msg.role === "user") return "我";
  if (msg.role === "spectator") return "S";
  return selectedProfile.value ? selectedProfile.value.name.charAt(0) : "AI";
}
function updateUrlSlug(slug) {
  const url = new URL(window.location.href);
  if (slug) {
    url.searchParams.set("slug", slug);
  } else {
    url.searchParams.delete("slug");
  }
  history.replaceState(null, "", url.toString());
}
function scrollToBottom() {
  nextTick(() => {
    if (messagesRef.value) {
      messagesRef.value.scrollTop = messagesRef.value.scrollHeight;
    }
  });
}
function focusInput() {
  nextTick(() => {
    if (inputRef.value) inputRef.value.focus();
  });
}
function resetSuggestionTimer() {
  if (suggestionTimer) {
    clearTimeout(suggestionTimer);
    suggestionTimer = null;
  }
}
function resetTransientState() {
  isTyping.value = false;
  pendingSuggestions.value = [];
  spectatorStatus.value = "Spectator 会在合适时机主动提示。";
  autopilotRunning.value = false;
  autopilotMode.value = "normal";
```

```
  autopilotTranscript.value = [];
  autopilotStatus.value = "尚未开始自动讨论";
  autopilotRound.value = 0;
  autopilotStopRequested.value = false;
}
function clearConversationState() {
  messages.value = [];
  inputText.value = "";
  autopilotReport.value = "";
  resetTransientState();
}
async function loadProfiles() {
  try {
    const res = await fetch("/api/profiles");
    const data = await res.json();
    if (!data.ok) return;
    profiles.value = data.data || [];
    if (!selectedSlug.value && profiles.value.length > 0) {
      const preferred = profiles.value.find((item) => item.slug === initialSlug);
      selectedSlug.value = preferred ? preferred.slug : profiles.value[0].slug;
      updateUrlSlug(selectedSlug.value);
    }
  } catch (e) {
    console.error("Failed to load profiles:", e);
  }
}
async function loadConversation() {
  if (!chatUserId.value || !selectedSlug.value) {
    clearConversationState();
    return;
  }
  resetSuggestionTimer();
  loadingConversation.value = true;
  resetTransientState();
  inputText.value = "";
  conversationStatus.value = `正在加载编号 ${chatUserId.value} 的独立记录...`;
  try {
    const res = await fetch(`/api/chat-users/${encodeURIComponent(chatUserId.value)}/conversations/${enc
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    messages.value = Array.isArray(data.data.messages) ? data.data.messages : [];
    autopilotReport.value = data.data.autopilotReport || "";
    conversationStatus.value = messages.value.length > 0 ? "已加载该编号下的历史记录。" : "当前编号下还没有
    scrollToBottom();
  } catch (e) {
    console.error("Load conversation error:", e);
    messages.value = [];
    autopilotReport.value = "";
```

```
    conversationStatus.value = `加载历史失败：${e.message || "network_error"}`;
  } finally {
    loadingConversation.value = false;
  }
}
async function persistConversationState() {
  if (!chatUserId.value || !selectedSlug.value || !selectedProfile.value) return;
  try {
    await fetch(`/api/chat-users/${encodeURIComponent(chatUserId.value)}/conversations/${encodeURICon
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        profileName: selectedProfile.value.name || "",
        messages: messages.value,
        autopilotReport: autopilotReport.value
      })
    });
  } catch (e) {
    console.error("Persist conversation error:", e);
  }
}
function setActiveUser(userId) {
  chatUserId.value = userId;
  loginInput.value = userId;
  if (userId) {
    localStorage.setItem(ACTIVE_CHAT_USER_KEY, userId);
    loginStatus.value = `当前编号：${userId}`;
  } else {
    localStorage.removeItem(ACTIVE_CHAT_USER_KEY);
    loginStatus.value = "未登录编号用户";
  }
}
async function allocateChatUser() {
  if (loginLoading.value) return;
  loginLoading.value = true;
  loginStatus.value = "正在分配新编号...";
  try {
    const res = await fetch("/api/chat-users/allocate", { method: "POST" });
    const data = await res.json();
    if (!res.ok || !data.ok || !data.data?.userId) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    setActiveUser(data.data.userId);
    conversationStatus.value = "已分配新编号，正在载入对应会话。";
    await loadConversation();
  } catch (e) {
    console.error("Allocate chat user error:", e);
    loginStatus.value = `分配失败：${e.message || "network_error"}`;
  } finally {
    loginLoading.value = false;
```

```
  }
}
async function loginChatUser() {
  if (loginLoading.value) return;
  const userId = loginInput.value.trim();
  if (!/^\d{10}$/.test(userId)) {
    loginStatus.value = "请输入 10 位编号。";
    return;
  }
  loginLoading.value = true;
  loginStatus.value = "正在登录编号用户...";
  try {
    const res = await fetch("/api/chat-users/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId })
    });
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    setActiveUser(userId);
    conversationStatus.value = "编号登录成功，正在载入对应会话。";
    await loadConversation();
  } catch (e) {
    console.error("Chat user login error:", e);
    loginStatus.value = `登录失败：${e.message || "network_error"}`;
  } finally {
    loginLoading.value = false;
  }
}
function logoutChatUser() {
  if (autopilotRunning.value) return;
  setActiveUser("");
  clearConversationState();
  conversationStatus.value = "已退出编号用户。";
}
async function restoreActiveUser() {
  const stored = localStorage.getItem(ACTIVE_CHAT_USER_KEY);
  if (!stored || !/^\d{10}$/.test(stored)) return;
  loginInput.value = stored;
  try {
    const res = await fetch("/api/chat-users/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId: stored })
    });
    const data = await res.json();
    if (!res.ok || !data.ok) {
      localStorage.removeItem(ACTIVE_CHAT_USER_KEY);
```

```
      return;
    }
    setActiveUser(stored);
    await loadConversation();
  } catch (e) {
    console.error("Restore active user error:", e);
  }
}
async function selectProfile(slug) {
  if (autopilotRunning.value || selectedSlug.value === slug) return;
  selectedSlug.value = slug;
  updateUrlSlug(slug);
  await loadConversation();
}
function dismissSuggestion(id) {
  pendingSuggestions.value = pendingSuggestions.value.filter((item) => item.id !== id);
}
function applySuggestion(item) {
  inputText.value = item.assistantPrompt || "";
  dismissSuggestion(item.id);
  spectatorStatus.value = "已将建议填入输入框。";
  focusInput();
}
async function sendSuggestionNow(item) {
  inputText.value = item.assistantPrompt || "";
  dismissSuggestion(item.id);
  await sendMessage();
}
async function consumeNdjsonResponse(res, onDelta) {
  const reader = res.body.getReader();
  const decoder = new TextDecoder();
  let buffer = "";
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    const lines = buffer.split("\n");
    buffer = lines.pop() || "";
    for (const line of lines) {
      if (!line.trim()) continue;
      const data = JSON.parse(line);
      if (!data.ok) {
        throw new Error(data.message || data.error || "unknown_error");
      }
      if (data.delta) onDelta(data.delta);
    }
  }
  if (buffer.trim()) {
    const data = JSON.parse(buffer);
    if (!data.ok) throw new Error(data.message || data.error || "unknown_error");
```

```
    if (data.delta) onDelta(data.delta);
  }
}
async function sendMessage() {
  if (!canInteract.value || !inputText.value.trim() || isTyping.value) return;
  const userText = inputText.value.trim();
  inputText.value = "";
  messages.value.push({ id: newId("msg"), role: "user", content: userText });
  scrollToBottom();
  pendingSuggestions.value = [];
  spectatorStatus.value = "等待模拟同事回复后再判断是否需要介入。";
  conversationStatus.value = "正在生成回复...";
  const assistantMessage = { id: newId("msg"), role: "assistant", content: "", label: selectedProfile.value ? sel
  isTyping.value = true;
  messages.value.push(assistantMessage);
  scrollToBottom();
  try {
    const history = messages.value
      .filter((m) => m.id !== assistantMessage.id)
      .map((m) => ({ role: m.role, content: m.content }));
    const res = await fetch("/api/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ slug: selectedSlug.value, messages: history, chatUserId: chatUserId.value })
    });
    if (!res.ok) throw new Error(`http_${res.status}`);
    await consumeNdjsonResponse(res, (delta) => {
      assistantMessage.content += delta;
      scrollToBottom();
    });
    await persistConversationState();
    spectatorStatus.value = "可继续对话，或让 Spectator 帮你推进。";
    conversationStatus.value = "本轮回复已保存到当前编号。";
    scheduleSpectatorSuggestion(false);
  } catch (e) {
    console.error("Chat error:", e);
    assistantMessage.content = `抱歉，发生了错误：${e.message || "network error"}`;
    conversationStatus.value = `发送失败：${e.message || "network_error"}`;
  } finally {
    isTyping.value = false;
  }
}
async function requestSpectatorSuggestion(manual = false) {
  if (!canInteract.value || spectatorLoading.value || autopilotRunning.value) return;
  if (!manual && pendingSuggestions.value.length >= 2) return;
  spectatorLoading.value = true;
  spectatorStatus.value = manual ? "Spectator 正在生成建议..." : "Spectator 正在观察当前对话...";
  try {
    const res = await fetch("/api/chat/spectator", {
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        slug: selectedSlug.value,
        messages: messages.value.map((item) => ({ role: item.role, content: item.content })),
        draftMessage: inputText.value,
        pendingSuggestions: pendingSuggestions.value,
        manual
      })
    });
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    const suggestion = data.data || {};
    if (!suggestion.shouldIntervene || !suggestion.assistantPrompt) {
      spectatorStatus.value = suggestion.reasoning || "本轮暂不需要介入。";
      return;
    }
    const duplicated = pendingSuggestions.value.some((item) => item.assistantPrompt === suggestion.ass
    if (!duplicated) {
      pendingSuggestions.value.unshift({
        id: newId("sug"),
        topic: suggestion.topic || "建议",
        userAsk: suggestion.userAsk || "建议进一步优化当前问题",
        assistantPrompt: suggestion.assistantPrompt,
        suggestionType: suggestion.suggestionType || "improvement",
        reasoning: suggestion.reasoning || ""
      });
      pendingSuggestions.value = pendingSuggestions.value.slice(0, 3);
      spectatorStatus.value = "Spectator 发现了一个值得介入的点。";
    }
  } catch (e) {
    console.error("Spectator suggestion error:", e);
    spectatorStatus.value = `建议生成失败：${e.message || "network_error"}`;
  } finally {
    spectatorLoading.value = false;
  }
}
function scheduleSpectatorSuggestion(manual = false) {
  resetSuggestionTimer();
  if (!canInteract.value || autopilotRunning.value) return;
  if (manual) {
    requestSpectatorSuggestion(true);
    return;
  }
  suggestionTimer = setTimeout(() => {
    requestSpectatorSuggestion(false);
  }, 1200);
}
function parseSpectatorProtocol(raw) {
```

```
  const match = raw.match(/^SHOULD_END=(true|false)\s*\nCONTENT:\s*\n?/i);
  if (!match) return { shouldEnd: false, content: raw };
  return { shouldEnd: match[1].toLowerCase() === "true", content: raw.slice(match[0].length) };
}
async function consumeTextStream(res, onChunk) {
  const reader = res.body.getReader();
  const decoder = new TextDecoder();
  let raw = "";
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    raw += decoder.decode(value, { stream: true });
    onChunk(raw);
  }
  return raw;
}
async function runAutopilotStep(nextSpeaker, stepIndex, token) {
  const chatMirrorMessage = {
    id: newId("msg"),
    role: nextSpeaker === "spectator" ? "spectator" : "assistant",
    content: "",
    label: nextSpeaker === "spectator" ? "Spectator · 自动讨论" : `${selectedProfile.value ? selectedProfile.val
  };
  messages.value.push(chatMirrorMessage);
  scrollToBottom();
  const item = { id: newId("auto"), speaker: nextSpeaker, content: "", streaming: true, chatMirrorId: chatMirr
  autopilotTranscript.value.push(item);
  const res = await fetch("/api/chat/autopilot/step", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      slug: selectedSlug.value,
      messages: messages.value.map((m) => ({ role: m.role, content: m.content })),
      transcript: autopilotTranscript.value
        .filter((entry) => entry.id !== item.id)
        .map((entry) => ({ speaker: entry.speaker, content: entry.content })),
      nextSpeaker,
      endRequested: autopilotStopRequested.value,
      mode: autopilotMode.value,
      stepIndex
    })
  });
  if (!res.ok) {
    const text = await res.text();
    throw new Error(text || `http_${res.status}`);
  }
  let shouldEnd = false;
  if (nextSpeaker === "spectator") {
    const raw = await consumeTextStream(res, (buffer) => {
      const parsed = parseSpectatorProtocol(buffer);
```

```
      item.content = parsed.content;
      chatMirrorMessage.content = parsed.content;
      scrollToBottom();
    });
    const parsed = parseSpectatorProtocol(raw);
    item.content = parsed.content;
    chatMirrorMessage.content = parsed.content;
    shouldEnd = parsed.shouldEnd;
  } else {
    const raw = await consumeTextStream(res, (buffer) => {
      item.content = buffer;
      chatMirrorMessage.content = buffer;
      scrollToBottom();
    });
    item.content = raw;
    chatMirrorMessage.content = raw;
  }
  item.streaming = false;
  await persistConversationState();
  if (token !== autopilotToken) throw new Error("autopilot_cancelled");
  return { shouldEnd };
}
async function generateAutopilotReport(token) {
  autopilotStatus.value = "正在生成自动讨论总结...";
  autopilotReport.value = "";
  const reportMessage = {
    id: newId("msg"),
    role: "spectator",
    content: "",
    label: "Spectator · 讨论总结"
  };
  messages.value.push(reportMessage);
  scrollToBottom();
  const res = await fetch("/api/chat/autopilot/report", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      slug: selectedSlug.value,
      messages: messages.value.map((m) => ({ role: m.role, content: m.content })),
      transcript: autopilotTranscript.value.map((entry) => ({ speaker: entry.speaker, content: entry.content })),
      mode: autopilotMode.value
    })
  });
  if (!res.ok) {
    const text = await res.text();
    throw new Error(text || `http_${res.status}`);
  }
  await consumeTextStream(res, (buffer) => {
    autopilotReport.value = buffer;
    reportMessage.content = buffer;
```

```
    scrollToBottom();
  });
  await persistConversationState();
  if (token === autopilotToken) autopilotStatus.value = "自动讨论已完成。";
}
async function startAutopilot(mode) {
  if (!canInteract.value || autopilotRunning.value) return;
  autopilotToken += 1;
  const token = autopilotToken;
  autopilotRunning.value = true;
  autopilotMode.value = mode;
  autopilotTranscript.value = [];
  autopilotReport.value = "";
  autopilotRound.value = 0;
  autopilotStopRequested.value = false;
  spectatorStatus.value = "自动讨论已启动，暂时暂停旁观建议。";
  conversationStatus.value = "自动讨论进行中...";
  let nextSpeaker = "main";
  try {
    for (let step = 1; step <= 6; step += 1) {
      if (token !== autopilotToken) break;
      autopilotRound.value = step;
      autopilotStatus.value = `${nextSpeaker === "main" ? selectedProfile.value.name : "Spectator"} 正在第 $
      const result = await runAutopilotStep(nextSpeaker, step, token);
      if (token !== autopilotToken) break;
      if (nextSpeaker === "spectator" && (result.shouldEnd || autopilotStopRequested.value || step === 6)) {
        await generateAutopilotReport(token);
        break;
      }
      nextSpeaker = nextSpeaker === "main" ? "spectator" : "main";
    }
  } catch (e) {
    if (e.message !== "autopilot_cancelled") {
      console.error("Autopilot error:", e);
      autopilotStatus.value = `自动讨论失败：${e.message || "unknown_error"}`;
      conversationStatus.value = `自动讨论失败：${e.message || "unknown_error"}`;
    }
  } finally {
    if (token === autopilotToken) {
      autopilotRunning.value = false;
      autopilotStopRequested.value = false;
      if (!autopilotReport.value && autopilotTranscript.value.length > 0 && !autopilotStatus.value.includes("失
        autopilotStatus.value = "自动讨论已结束。";
      }
      spectatorStatus.value = "自动讨论结束，Spectator 已恢复旁观。";
      if (!conversationStatus.value.includes("失败")) {
        conversationStatus.value = "自动讨论结果已保存到当前编号。";
      }
    }
  }
```

```
}
function requestAutopilotStop() {
  if (!autopilotRunning.value) return;
  autopilotStopRequested.value = true;
  autopilotStatus.value = "将在当前轮结束后收束并生成总结...";
}
function onKeyDown(e) {
  if (e.key === "Enter" && !e.shiftKey) {
    e.preventDefault();
    sendMessage();
  }
}
watch(inputText, (value) => {
  if (!canInteract.value || isTyping.value || autopilotRunning.value) return;
  if (!value.trim()) return;
  scheduleSpectatorSuggestion(false);
});
(async () => {
  await loadProfiles();
  await restoreActiveUser();
})();
return {
  profiles,
  selectedSlug,
  selectedProfile,
  messages,
  inputText,
  isTyping,
  messagesRef,
  inputRef,
  pendingSuggestions,
  spectatorLoading,
  spectatorStatus,
  autopilotRunning,
  autopilotReport,
  autopilotStatus,
  autopilotRound,
  chatUserId,
  loginInput,
  loginLoading,
  loginStatus,
  conversationStatus,
  loadingConversation,
  canInteract,
  renderMarkdown,
  messageClass,
  messageAvatar,
  selectProfile,
  allocateChatUser,
  loginChatUser,
```

```
        logoutChatUser,
        sendMessage,
        onKeyDown,
        requestSpectatorSuggestion,
        dismissSuggestion,
        applySuggestion,
        sendSuggestionNow,
        startAutopilot,
        requestAutopilotStop
      };
    }
  }).mount("#app");
 </script>
</body>
</html>
// ===== FILE END =====
// ===== FILE BEGIN: web-static\chat-admin.html =====
<!doctype html>
<html lang="zh-CN">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate" />
  <meta http-equiv="Pragma" content="no-cache" />
  <meta http-equiv="Expires" content="0" />
  <title>编号后台 - 会议旁听 Agent</title>
  <link rel="stylesheet" href="/style.css" />
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script src="/lib/marked.min.js"></script>
  <script src="/lib/purify.min.js"></script>
  <style>
   html,body { height:100%; overflow:hidden; }
   body { background:#F2F3F5; }
   #app.container { width:100vw; max-width:none; height:100vh; margin:0; padding:12px; display:flex; flex-direct
   .header { flex:0 0 auto; margin-bottom:12px; }
   .header-actions { display:flex; gap:10px; align-items:center; }
   .admin-layout { flex:1; min-height:0; display:grid; grid-template-columns:320px minmax(0, 380px) minmax(0,
   .panel { background:var(--card-bg); border:1px solid var(--border-color); border-radius:12px; min-height:0; di
   .panel-header { padding:12px 14px; border-bottom:1px solid var(--border-color); display:flex; justify-content:
   .panel-body { flex:1; min-height:0; overflow-y:auto; padding:12px 14px; display:flex; flex-direction:column; ga
   .user-card,.conversation-card,.message-card { background:var(--bg-gray); border:1px solid var(--border-color
   .profile-card { background:#F8FAFC; border:1px solid var(--border-color); border-radius:10px; padding:12px; }
   .user-card.active,.conversation-card.active { border-color:var(--primary-color); box-shadow:0 0 0 1px var(--pri
   .user-card,.conversation-card { cursor:pointer; }
   .card-title { font-size:14px; font-weight:600; }
   .card-meta,.hint-text { font-size:12px; color:var(--text-secondary); }
   .profile-meta-muted { margin-top:8px; font-size:11px; color:rgba(60, 60, 67, 0.5); }
   .profile-summary { font-size:13px; line-height:1.65; color:var(--text-color); }
   .tag-list { display:flex; flex-wrap:wrap; gap:6px; margin-top:6px; }
   .profile-tag { display:inline-flex; align-items:center; padding:2px 8px; border-radius:999px; background:#EEF2
```

```
    .profile-section-title { font-size:12px; font-weight:600; color:var(--text-secondary); margin-top:10px; }
    .message-card.user { background:#E8F0FE; border-color:#B6D4FE; }
    .message-card.spectator { background:#F4EEFF; border-color:#D9C9FF; }
    .message-label { font-size:12px; color:var(--text-secondary); margin-bottom:6px; }
    .message-content.md { background:transparent; border:none; padding:0; overflow:visible; max-height:none; }
    .empty-state { flex:1; display:flex; align-items:center; justify-content:center; text-align:center; color:var(--text-
    .status-text { font-size:12px; color:var(--text-secondary); }
    .btn.secondary,.btn { padding:7px 12px; }
    @media (max-width:1280px) {
      .admin-layout { grid-template-columns:1fr; }
    }
  </style>
</head>
<body>
  <div id="app" class="container">
    <div class="header">
      <div>
        <div class="title">编号后台</div>
        <div class="subtle">查看有哪些编号用户，以及每个编号下的同事画像对话记录</div>
      </div>
      <div class="header-actions">
        <button class="btn secondary" @click="loadUsers" :disabled="loadingUsers">{{ loadingUsers ? "刷新中..."
        <a href="/chat.html" class="btn secondary">进入对话</a>
        <a href="/" class="btn secondary">返回首页</a>
      </div>
    </div>
    <div class="admin-layout">
      <div class="panel">
        <div class="panel-header">
          <div class="panel-title">编号用户</div>
          <div class="status-text">{{ users.length }} 个</div>
        </div>
        <div class="panel-body">
          <div v-if="users.length === 0" class="empty-state">暂无编号用户</div>
          <div v-for="user in users" :key="user.userId" :class="['user-card', { active: selectedUserId === user.userId
            <div class="card-title">{{ user.userId }}</div>
            <div class="card-meta" style="margin-top:4px;">会话 {{ user.conversationCount }} 个</div>
            <div class="card-meta">最近更新 {{ formatTime(user.updatedAt) }}</div>
          </div>
        </div>
      </div>
      <div class="panel">
        <div class="panel-header">
          <div class="panel-title">画像会话</div>
          <div class="status-text">{{ selectedUserDetail ? selectedUserDetail.conversationCount : 0 }} 个</div>
        </div>
        <div class="panel-body">
          <div v-if="!selectedUser" class="empty-state">先选择一个编号用户</div>
          <div v-else-if="loadingUserDetail" class="empty-state">正在读取该编号的文件记录...</div>
          <div v-else-if="!selectedUserDetail || selectedUserDetail.conversations.length === 0" class="empty-state
```

```
<template v-else>
  <div class="profile-card">
    <div style="display:flex;justify-content:space-between;align-items:center;gap:8px;">
      <div class="card-title">用户画像</div>
      <div style="display:flex;gap:8px;align-items:center;">
        <button class="btn secondary" @click="toggleUserProfileExpanded" :disabled="loadingUserProfile
        <button class="btn secondary" @click="refreshUserProfile" :disabled="loadingUserProfile">{{ loadin
      </div>
    </div>
    <div v-if="loadingUserProfile" class="card-meta" style="margin-top:10px;">正在分析用户画像...</div>
    <template v-else-if="selectedUserProfile">
      <div class="profile-meta-muted">用户消息 {{ selectedUserProfile.stats?.totalUserMessages || 0 }} 条 · 会
      <template v-if="userProfileExpanded">
        <div class="profile-summary" style="margin-top:8px;">{{ selectedUserProfile.profile?.summary || '暂
        <div class="profile-section-title">沟通风格</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.communicationStyle || [])" :key="`c-${item}`" class
          <span v-if="!(selectedUserProfile.profile?.communicationStyle || []).length" class="card-meta">暂无
        </div>
        <div class="profile-section-title">技术关注</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.technicalFocus || [])" :key="`t-${item}`" class="pro
          <span v-if="!(selectedUserProfile.profile?.technicalFocus || []).length" class="card-meta">暂无</spa
        </div>
        <div class="profile-section-title">协作方式</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.collaborationStyle || [])" :key="`co-${item}`" class=
          <span v-if="!(selectedUserProfile.profile?.collaborationStyle || []).length" class="card-meta">暂无</
        </div>
        <div class="profile-section-title">决策模式</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.decisionPattern || [])" :key="`d-${item}`" class="pr
          <span v-if="!(selectedUserProfile.profile?.decisionPattern || []).length" class="card-meta">暂无</sp
        </div>
        <div class="profile-section-title">常见担忧</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.concerns || [])" :key="`r-${item}`" class="profile-ta
          <span v-if="!(selectedUserProfile.profile?.concerns || []).length" class="card-meta">暂无</span>
        </div>
        <div class="profile-section-title">目标偏好</div>
        <div class="tag-list">
          <span v-for="item in (selectedUserProfile.profile?.goals || [])" :key="`g-${item}`" class="profile-tag"
          <span v-if="!(selectedUserProfile.profile?.goals || []).length" class="card-meta">暂无</span>
        </div>
      </template>
    </template>
    <div v-else class="card-meta" style="margin-top:10px;">暂无用户画像。</div>
  </div>
  <div v-for="conversation in selectedUserDetail.conversations" :key="conversation.slug" :class="['conver
    <div class="card-title">{{ conversation.profileName || conversation.slug }}</div>
```

```
          <div class="card-meta" style="margin-top:4px;">标识 {{ conversation.slug }}</div>
          <div class="card-meta">消息 {{ conversation.messages ? conversation.messages.length : (conversation
          <div class="card-meta">最近更新 {{ formatTime(conversation.updatedAt) }}</div>
          <div class="card-meta" v-if="conversation.lastMessagePreview || lastMessagePreview(conversation)">
        </div>
      </template>
    </div>
  </div>
  <div class="panel">
    <div class="panel-header">
      <div class="panel-title">对话记录</div>
      <div class="status-text">{{ selectedConversation ? `${selectedConversation.messages?.length || 0} 条消息`
    </div>
    <div class="panel-body">
      <div v-if="loadingUserDetail" class="empty-state">正在读取会话文件...</div>
      <div v-else-if="!selectedConversation" class="empty-state">选择左侧会话查看完整记录</div>
      <template v-else>
        <div class="conversation-card">
          <div class="card-title">{{ selectedConversation.profileName || selectedConversation.slug }}</div>
          <div class="card-meta" style="margin-top:4px;">编号 {{ selectedUserId }}</div>
          <div class="card-meta">创建 {{ formatTime(selectedConversation.createdAt) }} · 更新 {{ formatTime(sele
        </div>
        <div v-for="msg in selectedConversation.messages" :key="msg.id || `${msg.role}-${msg.content.slice(0, 1
          <div v-if="msg.label && msg.role !== 'user'" class="message-label">{{ msg.label }}</div>
          <div class="card-meta" style="margin-bottom:6px;">{{ roleText(msg.role) }}</div>
          <div v-if="msg.role === 'user'">{{ msg.content }}</div>
          <div v-else class="message-content md" v-html="renderMarkdown(msg.content)"></div>
        </div>
        <div v-if="selectedConversation.autopilotReport" class="conversation-card">
          <div class="card-title">自动讨论总结</div>
          <div class="message-content md" style="margin-top:8px;" v-html="renderMarkdown(selectedConvers
        </div>
      </template>
    </div>
  </div>
 </div>
</div>
<script>
 const { createApp, ref, computed } = Vue;
 createApp({
  setup() {
    const users = ref([]);
    const selectedUserId = ref("");
    const selectedConversationSlug = ref("");
    const loadingUsers = ref(false);
    const loadingUserDetail = ref(false);
    const selectedUserDetail = ref(null);
    const loadingUserProfile = ref(false);
    const selectedUserProfile = ref(null);
    const userProfileExpanded = ref(false);
```

```
const selectedUser = computed(() => users.value.find((item) => item.userId === selectedUserId.value) || n
const selectedConversation = computed(() => {
  if (!selectedUserDetail.value) return null;
  return selectedUserDetail.value.conversations.find((item) => item.slug === selectedConversationSlug.valu
});
function renderMarkdown(text) {
  if (!text) return "";
  return DOMPurify.sanitize(marked.parse(text));
}
function formatTime(value) {
  if (!value) return "-";
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return value;
  return date.toLocaleString("zh-CN", { hour12: false });
}
function roleText(role) {
  if (role === "user") return "用户";
  if (role === "spectator") return "Spectator";
  return "模拟同事";
}
function lastMessagePreview(conversation) {
  const lastMessage = Array.isArray(conversation?.messages) ? conversation.messages[conversation.messag
  return lastMessage?.content ? lastMessage.content.slice(0, 120) : "";
}
async function loadUsers() {
  loadingUsers.value = true;
  try {
    const res = await fetch("/api/chat-admin/users");
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    users.value = Array.isArray(data.data) ? data.data : [];
    if (!selectedUserId.value && users.value.length > 0) {
      selectedUserId.value = users.value[0].userId;
    }
    if (selectedUserId.value) {
      await loadUserDetail(selectedUserId.value);
      await loadUserProfile(selectedUserId.value, false);
    }
  } catch (e) {
    console.error("Load admin users error:", e);
  } finally {
    loadingUsers.value = false;
  }
}
async function loadUserDetail(userId) {
  if (!userId) {
    selectedUserDetail.value = null;
    selectedConversationSlug.value = "";
```

```
    return;
  }
  loadingUserDetail.value = true;
  try {
    const res = await fetch(`/api/chat-admin/users/${encodeURIComponent(userId)}`);
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    selectedUserDetail.value = data.data || null;
    if (selectedUserDetail.value?.conversations?.length > 0) {
      const exists = selectedUserDetail.value.conversations.some((item) => item.slug === selectedConversat
      selectedConversationSlug.value = exists ? selectedConversationSlug.value : selectedUserDetail.value.co
    } else {
      selectedConversationSlug.value = "";
    }
  } catch (e) {
    console.error("Load admin user detail error:", e);
    selectedUserDetail.value = null;
    selectedConversationSlug.value = "";
  } finally {
    loadingUserDetail.value = false;
  }
}
async function loadUserProfile(userId, force = false) {
  if (!userId) {
    selectedUserProfile.value = null;
    userProfileExpanded.value = false;
    return;
  }
  loadingUserProfile.value = true;
  try {
    const suffix = force ? "?force=true" : "";
    const res = await fetch(`/api/chat-admin/users/${encodeURIComponent(userId)}/profile${suffix}`);
    const data = await res.json();
    if (!res.ok || !data.ok) {
      throw new Error(data.message || data.error || `http_${res.status}`);
    }
    selectedUserProfile.value = data.data || null;
    userProfileExpanded.value = false;
  } catch (e) {
    console.error("Load admin user profile error:", e);
    selectedUserProfile.value = null;
    userProfileExpanded.value = false;
  } finally {
    loadingUserProfile.value = false;
  }
}
async function selectUser(userId) {
  selectedUserId.value = userId;
```

```
        await loadUserDetail(userId);
        await loadUserProfile(userId, false);
      }
      async function refreshUserProfile() {
        if (!selectedUserId.value) return;
        await loadUserProfile(selectedUserId.value, true);
      }
      function toggleUserProfileExpanded() {
        if (!selectedUserProfile.value) return;
        userProfileExpanded.value = !userProfileExpanded.value;
      }
      function selectConversation(slug) {
        selectedConversationSlug.value = slug;
      }
      loadUsers();
      return {
        users,
        selectedUserId,
        selectedConversationSlug,
        selectedUser,
        selectedConversation,
        loadingUsers,
        loadingUserDetail,
        selectedUserDetail,
        loadingUserProfile,
        selectedUserProfile,
        userProfileExpanded,
        renderMarkdown,
        formatTime,
        roleText,
        lastMessagePreview,
        loadUsers,
        loadUserDetail,
        loadUserProfile,
        selectUser,
        selectConversation,
        refreshUserProfile,
        toggleUserProfileExpanded
      };
    }
  }).mount("#app");
</script>
</body>
</html>
// ===== FILE END =====
// ===== FILE BEGIN: web-static\index.html =====
<!doctype html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta http-equiv="Cache-Control" content="no-cache, no-store, must-revalidate" />
    <meta http-equiv="Pragma" content="no-cache" />
    <meta http-equiv="Expires" content="0" />
    <title>会议旁听 Agent</title>
    <link rel="stylesheet" href="/style.css" />
    <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
    <script src="/lib/marked.min.js"></script>
    <script src="/lib/purify.min.js"></script>
  </head>
  <body>
    <div id="app"></div>
    <script src="/app.js?v=8"></script>
  </body>
</html>
// ===== FILE END =====
// ===== FILE BEGIN: web-static\lib\marked.min.js =====
/**
 * marked v12.0.0 - a markdown parser
 * Copyright (c) 2011-2024, Christopher Jeffrey. (MIT Licensed)
 * https://github.com/markedjs/marked
 */
!function(e,t){"object"==typeof exports&&"undefined"!=typeof module?t(exports):"function"==typeof define&&
// ===== FILE END =====
// ===== FILE BEGIN: web-static\lib\purify.min.js =====
/*! @license DOMPurify 3.0.9 | (c) Cure53 and other contributors | Released under the Apache license 2.0 and Mo
!function(e,t){"object"==typeof exports&&"undefined"!=typeof module?module.exports=t():"function"==typeof
//# sourceMappingURL=purify.min.js.map
// ===== FILE END =====
// ===== FILE BEGIN: web-static\style.css =====
:root {
  font-family: "PingFang SC", "Microsoft YaHei", "Helvetica Neue", Helvetica, Arial, sans-serif;
  color: #111111;
  background: #F2F3F5;
  --primary-color: #006EFF;
  --primary-hover: #0052D9;
  --danger-color: #E34D59;
  --border-color: #E5E6EB;
  --card-bg: #FFFFFF;
  --text-secondary: #4E5969;
  --text-placeholder: #86909C;
  --bg-gray: #F7F8FA;
}
* {
  box-sizing: border-box;
}
body {
  margin: 0;
  -webkit-font-smoothing: antialiased;
}
```

```
.container {
  max-width: 1080px;
  margin: 0 auto;
  padding: 24px 16px 80px;
}
.header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 20px;
}
.title {
  font-size: 24px;
  font-weight: 600;
  color: #111111;
}
.subtle {
  font-size: 13px;
  color: var(--text-secondary);
}
.card {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 20px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.04);
  margin-bottom: 16px;
  max-width: 100%;
}
.row {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
  align-items: center;
}
.btn {
  appearance: none;
  border: 1px solid transparent;
  background: var(--primary-color);
  color: #fff;
  padding: 8px 16px;
  border-radius: 4px;
  font-size: 14px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s;
}
.btn:hover:not(:disabled) {
  background: var(--primary-hover);
```

```
}
.btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
.btn.secondary {
  background: var(--bg-gray);
  color: #111111;
  border: 1px solid var(--border-color);
}
.btn.secondary:hover:not(:disabled) {
  background: var(--border-color);
}
.panels {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
  margin-top: 16px;
}
.panel-title {
  font-weight: 600;
  font-size: 16px;
  margin-bottom: 12px;
  color: #111111;
  display: flex;
  align-items: center;
}
.panel-title::before {
  content: "";
  display: inline-block;
  width: 4px;
  height: 16px;
  background: var(--primary-color);
  margin-right: 8px;
  border-radius: 2px;
}
.pre {
  white-space: pre-wrap;
  word-break: break-word;
  overflow-wrap: anywhere;
  line-height: 1.6;
  font-size: 14px;
  color: #111111;
  background: var(--bg-gray);
  border: 1px solid var(--border-color);
  border-radius: 6px;
  padding: 16px;
  max-height: 60vh;
  overflow-y: auto;
  overflow-x: hidden;
```

```
  max-width: 100%;
}
.md {
  word-break: break-word;
  overflow-wrap: anywhere;
  line-height: 1.6;
  font-size: 15px;
  color: #333333;
  /* 腾讯会议主题背景：浅雅的蓝灰底色 */
  background: #F4F8FF;
  border: 1px solid #D6E4FF;
  border-radius: 8px;
  padding: 16px 20px;
  max-height: 60vh;
  overflow-y: auto;
  overflow-x: hidden;
  max-width: 100%;
}
.md :first-child {
  margin-top: 0;
}
.md :last-child {
  margin-bottom: 0;
}
.md h1,
.md h2,
.md h3,
.md h4,
.md h5 {
  margin: 1.5em 0 0.8em;
  color: #111111;
  font-weight: 600;
  line-height: 1.25;
}
.md h1 { font-size: 1.6em; padding-bottom: 0.3em; border-bottom: 1px solid var(--border-color); }
.md h2 { font-size: 1.4em; padding-bottom: 0.3em; border-bottom: 1px solid var(--border-color); }
.md h3 { font-size: 1.2em; }
.md h4 { font-size: 1.1em; }
.md p {
  margin: 1em 0;
}
.md ul,
.md ol {
  margin: 1em 0;
  padding-left: 2em;
}
.md li {
  margin: 0.3em 0;
}
.md li > p {
```

```
  margin: 0.3em 0;
}
.md strong {
  font-weight: 600;
  color: #000;
}
.md em {
  font-style: italic;
}
.md code {
  font-family: Consolas, Monaco, "Courier New", monospace;
  font-size: 0.9em;
  background: rgba(0, 0, 0, 0.05);
  color: var(--danger-color);
  padding: 0.2em 0.4em;
  border-radius: 3px;
  word-break: break-word;
  white-space: pre-wrap;
}
.md pre {
  background: transparent;
  color: inherit;
  padding: 0;
  border-radius: 0;
  overflow-x: auto;
  overflow-y: hidden;
  white-space: pre-wrap;
  word-break: break-word;
  overflow-wrap: anywhere;
  margin: 1em 0;
}
.md pre code {
  background: transparent;
  color: inherit;
  padding: 0;
  font-size: 14px;
  white-space: pre-wrap;
  word-break: break-word;
}
.md blockquote {
  margin: 1em 0;
  padding: 0.5em 1em;
  border-left: 4px solid var(--primary-color);
  background: var(--bg-gray);
  color: var(--text-secondary);
}
.md blockquote p {
  margin: 0;
}
.md table {
```

```
  border-collapse: collapse;
  width: 100%;
  margin: 1em 0;
  display: block;
  overflow-x: auto;
  word-break: normal;
}
.md th,
.md td {
  border: 1px solid var(--border-color);
  padding: 10px 14px;
  vertical-align: top;
  font-size: 14px;
}
.md th {
  background: var(--bg-gray);
  font-weight: 600;
}
.md a {
  color: var(--primary-color);
  text-decoration: none;
}
.md a:hover {
  text-decoration: underline;
}
.md hr {
  height: 1px;
  padding: 0;
  margin: 24px 0;
  background-color: var(--border-color);
  border: 0;
}
.error {
  color: var(--danger-color);
  background: #FFEBEB;
  border: 1px solid #FFD1D1;
  border-radius: 6px;
  padding: 12px 16px;
  margin-top: 12px;
  font-size: 14px;
}
.hint {
  font-size: 13px;
  color: var(--text-placeholder);
  margin-top: 8px;
}
.monitor {
  margin-top: 16px;
}
.floating-ball {
```

```
  position: fixed;
  right: 32px;
  bottom: 32px;
  width: 56px;
  height: 56px;
  background-color: var(--primary-color);
  color: white;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  box-shadow: 0 4px 12px rgba(0, 110, 255, 0.3);
  cursor: pointer;
  font-weight: 500;
  font-size: 14px;
  user-select: none;
  z-index: 9999;
  transition: all 0.2s ease;
}
.floating-ball:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 16px rgba(0, 110, 255, 0.4);
}
.floating-ball.capturing {
  background-color: var(--danger-color);
  box-shadow: 0 4px 12px rgba(227, 77, 89, 0.3);
  animation: pulse 2s infinite;
}
@keyframes pulse {
  0% {
    box-shadow: 0 0 0 0 rgba(227, 77, 89, 0.4);
  }
  70% {
    box-shadow: 0 0 0 12px rgba(227, 77, 89, 0);
  }
  100% {
    box-shadow: 0 0 0 0 rgba(227, 77, 89, 0);
  }
}
.btn-sm {
  appearance: none;
  border: 1px solid var(--border-color);
  background: var(--bg-gray);
  color: var(--text-secondary);
  padding: 4px 10px;
  border-radius: 4px;
  font-size: 12px;
  cursor: pointer;
  transition: all 0.2s;
}
```

```
.btn-sm:hover:not(:disabled) {
  background: var(--border-color);
  color: #111;
}
.btn-sm:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}
.speaker-bar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
  padding: 10px 0 6px;
  border-bottom: 1px solid var(--border-color);
  margin-bottom: 10px;
}
.speaker-bar-label {
  font-size: 12px;
  color: var(--text-placeholder);
  font-weight: 500;
}
.speaker-chip {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  padding: 4px 10px;
  background: var(--bg-gray);
  border: 1px solid var(--border-color);
  border-radius: 14px;
  font-size: 13px;
  color: #111;
  cursor: pointer;
  transition: all 0.15s;
  white-space: nowrap;
}
.speaker-chip:hover {
  background: #E8F0FE;
  border-color: #B6D4FE;
}
.rename-hint {
  font-size: 10px;
  color: var(--text-placeholder);
  margin-left: 2px;
}
.speaker-input {
  width: 80px;
  padding: 2px 6px;
  border: 1px solid var(--primary-color);
  border-radius: 4px;
```

```
  font-size: 13px;
  outline: none;
  background: #fff;
}
.chip-btn {
  appearance: none;
  border: none;
  background: none;
  cursor: pointer;
  font-size: 14px;
  padding: 0 2px;
  color: var(--text-secondary);
  line-height: 1;
}
.chip-btn:first-of-type:hover {
  color: #07C160;
}
.chip-btn:last-of-type:hover {
  color: var(--danger-color);
}
.profile-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
  gap: 12px;
  margin: 12px 0;
}
.profile-card {
  position: relative;
  background: var(--bg-gray);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 14px;
  cursor: pointer;
  transition: all 0.15s;
}
.profile-card:hover {
  background: #E8F0FE;
  border-color: #B6D4FE;
}
.profile-card.active {
  background: #E8F0FE;
  border-color: var(--primary-color);
  box-shadow: 0 0 0 1px var(--primary-color);
}
.profile-name {
  font-weight: 600;
  font-size: 15px;
  color: #111;
  margin-bottom: 4px;
}
```

```
.profile-role {
  font-size: 12px;
  color: var(--text-secondary);
  margin-bottom: 8px;
}
.profile-tags {
  display: flex;
  flex-wrap: wrap;
  gap: 4px;
  margin-bottom: 6px;
}
.tag {
  display: inline-block;
  padding: 2px 8px;
  background: #E8F0FE;
  color: var(--primary-color);
  border-radius: 10px;
  font-size: 11px;
  font-weight: 500;
}
.profile-impression {
  font-size: 12px;
  color: var(--text-secondary);
  font-style: italic;
  margin-bottom: 4px;
  line-height: 1.4;
}
.profile-meta {
  font-size: 11px;
  color: var(--text-placeholder);
}
.btn-delete-sm {
  position: absolute;
  top: 6px;
  right: 6px;
  appearance: none;
  border: none;
  background: none;
  color: var(--text-placeholder);
  cursor: pointer;
  font-size: 16px;
  line-height: 1;
  padding: 2px 4px;
  border-radius: 4px;
  transition: all 0.15s;
}
.btn-delete-sm:hover {
  color: var(--danger-color);
  background: #FFEBEB;
}
```

```
.profile-detail {
  margin-top: 16px;
  border-top: 1px solid var(--border-color);
  padding-top: 16px;
}
.profile-detail-header {
  margin-bottom: 16px;
}
.profile-detail-header h3 {
  font-size: 18px;
  font-weight: 600;
  color: #111;
  margin: 0;
}
.profile-section {
  margin-bottom: 16px;
}
.profile-section h4 {
  font-size: 14px;
  font-weight: 600;
  color: var(--text-secondary);
  margin: 0 0 8px;
  padding-bottom: 4px;
  border-bottom: 1px dashed var(--border-color);
}
// ===== FILE END =====
```